Лабораторная работа №5. Линейная регрессия с L1-регуляризацией: SIMD и OpenMP

Выполнил: Москаленко Дмитрий, группа 11-401

1. Параметры экспериментов и система

Процессор Intel Core i5-12450H: 8 ядер (12 логических процессоров), кеш L1 - 704 КБ, L2 - 7 МБ, L3 - 12 МБ; компилятор GCC 16.2.0, флаги -O3 -march=native -fopenmp (кроме эксперимента 3.7). Данные (double) генерируются с фиксированным seed: признаки - случайные числа в [−1; 1], y = 2·(каждый 4-й признак) + 0,5 + шум. По умолчанию $N = 10^5$, d = 32, K = 100, $\eta = 0{,}5$, $\lambda = 10^{-3}$, 8 потоков, начальные w = 0, b = 0. Время - медиана из 5 запусков, в замер входит только обучение; допуск при проверке корректности - 10^{-9}.

2. Результаты

Таблица 1. Проверка корректности ($N = 10^4$, d = 32)

Реализация	Итоговая F(w, b)	Δw	Корректность
Последовательная	0,0176507721	0	эталон
OpenMP	0,0176507721	$2{,}2\cdot10^{-16}$	да
SIMD	0,0176507721	$2{,}2\cdot10^{-16}$	да
OpenMP + SIMD	0,0176507721	$2{,}2\cdot10^{-16}$	да

Таблица 2. Сравнение реализаций ($N = 10^5$, d = 32, 8 потоков)

Вариант	Время, с	Ускорение S
Последовательная	0,221	1,00
OpenMP	0,081	2,73
SIMD	0,197	1,12
OpenMP + SIMD	0,065	3,40

Таблица 3. Влияние числа объектов (d = 32, 8 потоков)

N	serial, с	OpenMP, с	SIMD, с	OpenMP + SIMD, с
10^4	0,011	0,007	0,007	0,007
10^5	0,221	0,081	0,197	0,065
10^6	2,405	0,723	2,142	0,741

Таблица 4. Влияние числа признаков ($N = 10^5$, 8 потоков)

d	serial, с	OpenMP, с	SIMD, с	OpenMP + SIMD, с	serial / SIMD
8	0,042	0,016	0,046	0,017	0,91
32	0,221	0,081	0,197	0,065	1,12
128	0,821	0,249	0,627	0,252	1,31
256	2,255	0,521	2,067	0,552	1,09

Таблица 5. Влияние числа потоков ($N = 10^5$, d = 32; S и E - относительно 1 потока той же версии)

p	OpenMP, с	S OpenMP	E OpenMP	OpenMP + SIMD, с	S OMP+SIMD	E OMP+SIMD
1	0,192	1,00	1,00	0,168	1,00	1,00
2	0,115	1,67	0,83	0,109	1,54	0,77
4	0,074	2,59	0,65	0,064	2,63	0,66
8	0,081	2,37	0,30	0,065	2,58	0,32
16	0,060	3,20	0,20	0,054	3,11	0,19

Таблица 6. Влияние коэффициента регуляризации ($N = 10^5$, d = 32, 8 потоков)

λ	Итоговая F(w, b)	Число $\lvert w_j\rvert < 10^{-6}$ (из 32)	Время OMP + SIMD, с
0	0,0016648	0	0,058
10^{-4}	0,0032651	15	0,061
10^{-3}	0,0176534	24	0,065
10^{-2}	0,1604665	24	0,058
10^{-1}	1,4817472	24	0,055

Таблица 7. Оптимизации компилятора ($N = 10^5$, d = 32)

Флаги компиляции	Время, с	Ускорение	Примечание
-O0 -fopenmp	1,145	1,00	serial, без оптимизации
-O3 -fopenmp	0,212	5,40	serial, векторы SSE2 (16 байт)
-O3 -march=native -fopenmp	0,179	6,40	serial, векторы AVX2 (32 байта)
-O3 -march=native -fopenmp + omp simd	0,160	7,16	версия SIMD

3. Выводы

1. Самая быстрая реализация - OpenMP + SIMD: при $N = 10^5$ и $d = 32$ она в 3,4 раза быстрее последовательной. При больших N и d её преимущество перед OpenMP пропадает: при $N = 10^6$, а также $d \geq 128$ обе работают почти одинаково, потому что упираются в память.

2. На 8 потоках OpenMP ускоряет программу в 2,7 раза, OpenMP + SIMD - в 3,4 раза. Ускорение меньше числа потоков, потому что матрица X (25,6 МБ) не помещается в кеш L3 (12 МБ) и на каждой итерации читается из оперативной памяти, а её пропускная способность общая для всех ядер.

3. Эффективность падает с 0,83 на 2 потоках до 0,30 на 8 и 0,20 на 16. После 4 потоков время почти не уменьшается - это насыщение пропускной способности памяти (около 35 ГБ/с при $N = 10^6$), а не рост накладных расходов.

4. Компилятор сам векторизовал оба цикла по признакам: накопление градиента и сумму прогноза pred. Сумму pred он векторизовал без перестановки сложений, так как для дробных чисел порядок сложения влияет на результат. Директива omp simd reduction(+: pred) разрешает перестановку, и версия SIMD стала быстрее на 11%. Не векторизованы внешний цикл по объектам (внутри него два цикла подряд), цикл обновления весов (ветвления в soft) и генерация данных (вызовы rand). Выгода SIMD растёт с числом признаков: при $d = 8$ её нет, при $d = 128$ ускорение 1,31 раза, а при $d = 256$ оно снова меньше из-за памяти. Основное ускорение дают сами флаги -O3 (в 5,4 раза) и -march=native.

5. Чем больше λ, тем сильнее коэффициенты прижимаются к нулю: при $\lambda = 0$ нулевых нет, при $\lambda = 10^{-4}$ их 15, а при $\lambda \geq 10^{-3}$ остаются только 8 признаков из 32 - ровно те, по которым генерировались данные. Функция потерь при этом растёт, потому что веса уменьшаются и добавляется штраф $\lambda \cdot \Sigma |w_j|$.

6. Главное ограничение - доступ к памяти: на каждой итерации вся матрица X читается заново, а вычислений на один элемент мало. При малых N (10^4) заметны накладные расходы на запуск потоков. Объединение частичных градиентов занимает мало времени.